

¿Qué técnicas fueron más efectivas para reducir el uso de memoria?

a. Muestreo Monte Carlo

- En lugar de procesar todos los bloques, se seleccionó aleatoriamente un porcentaje (10%).
- Esto redujo drásticamente el uso de memoria y tiempo, manteniendo una estimación razonable del comportamiento del sistema completo.
- Es especialmente útil cuando el sistema tiene comportamientos estadísticos homogéneos o se busca una solución aproximada.

b. Uso de matrices dispersas (scipy.sparse)

- Representar la matriz en formato disperso (csr_matrix) en lugar de numpy.ndarray redujo el uso de RAM de manera exponencial, sobre todo para matrices grandes.
- Esto permite manejar estructuras que serían imposibles de almacenar densamente en memoria.

c. Procesamiento por bloques

- Al dividir el espacio de estados en bloques, se evitó cargar toda la matriz en memoria.
- Solo se cargaron en memoria los bloques necesarios, reduciendo consumo y facilitando la paralelización.

¿Qué desafíos se encontraron al implementar las estrategias?

a. Problemas numéricos (overflow, NaN)

- Algunos bloques causaban errores de cálculo por tener valores extremos → se solucionó normalizando los datos y usando límites (Bounds) en la optimización.

b. Distribución de datos no uniforme

- Muchos bloques estaban vacíos (nnz = 0), lo que implicaba procesamiento innecesario si no se filtraban previamente.
- Se tuvo que aplicar lógica para ignorar bloques vacíos y balancear la carga de trabajo.

c. Sincronización de procesos

- En multiprocessing, cada proceso usa su propia memoria, así que es importante no compartir objetos grandes innecesariamente.
- Usar map() con argumentos ligeros fue clave.

d. Estimación vs precisión

- Monte Carlo ofrece eficiencia, pero puede variar su precisión dependiendo de la muestra.
- En sistemas muy heterogéneos, puede ser necesario aumentar la fracción de muestra o usar muestreo estratificado.

Discusión sobre el uso de hardware especializado

a. Uso de GPU (CUDA, cuBLAS, cuSparse)

- Las GPUs son ideales para operaciones vectorizadas, como:
 - Multiplicación de matrices
 - Simulación de muchas partículas o estados
- Herramientas como CuPy, PyTorch, o TensorFlow pueden reemplazar NumPy para trabajar en GPU.
- En matrices dispersas, cupy.sparse permite acelerar operaciones hasta 100x en comparación con CPU.

b. Uso de clústeres de computación (HPC / Cloud)

- Para problemas realmente grandes (millones de estados), usar un clúster distribuido permite:
 - Dividir bloques entre nodos
 - Ejecutar procesamiento paralelo a escala

Herramientas recomendadas:

- Dask (paralelismo distribuido en Python)
- Spark (procesamiento distribuido de datos)
- Cloud: AWS Batch, GCP Dataproc, Azure Batch